

SPRING DATA JPA

Cheat Sheet Ultime & Guide de Référence Pratique

Modèle d'Entité Référence

Toutes les requêtes de ce document sont basées sur l'entité JPA standardisée suivante :

```
@Entity
@Table(name = "product")
public class Product {
    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    private String name;
    private BigDecimal price;
    private Integer stockQuantity;
    private Boolean active;
    private String brand;

    // Getters, Setters, Constructeurs
}
```

1. Méthodes de Requêtes Dérivées (Query Methods)

Générées automatiquement par Spring Data JPA à partir de la signature de la méthode dans l'interface de votre Repository.

Recherche simple

JAVA REPOSITORY

```
Product findByName(String name);
```

SQL ÉQUIVALENT

```
SELECT * FROM product WHERE name = ?;
```

Plusieurs conditions

JAVA REPOSITORY

```
Product findByNameAndPrice(String name,
    BigDecimal price);
List<Product> findByNameOrPrice(String name,
    BigDecimal price);
```

SQL ÉQUIVALENT

```
WHERE name = ? AND price = ?;
WHERE name = ? OR price = ?;
```

Comparaisons Numériques

JAVA REPOSITORY

```
List<Product> findByPriceGreaterThan(BigDecimal
    price);
List<Product> findByPriceLessThan(BigDecimal
    price);
List<Product> findByPriceBetween(BigDecimal
    min, BigDecimal max);
```

SQL ÉQUIVALENT

Recherche Textuelle

JAVA REPOSITORY

```
List<Product> findByNameContaining(String
    keyword);
List<Product> findByNameStartingWith(String
    prefix);
List<Product> findByNameEndingWith(String
    suffix);
List<Product> findByNameIgnoreCase(String
    name);
```

SQL ÉQUIVALENT

```
WHERE price > ?
WHERE price < ?
WHERE price BETWEEN ? AND ?
```

```
WHERE name LIKE %keyword%
WHERE name LIKE 'prefix%'
WHERE name LIKE '%suffix'
WHERE LOWER(name) = LOWER(?)
```

Valeurs Booléennes & Nullité

JAVA REPOSITORY

```
List<Product> findByActiveTrue();
List<Product> findByActiveFalse();
List<Product> findByBrandIsNull();
List<Product> findByBrandIsNotNull();
```

SQL ÉQUIVALENT

```
WHERE active = true
WHERE active = false
WHERE brand IS NULL
WHERE brand IS NOT NULL
```

Tri, Restriction & Agrégations

JAVA REPOSITORY

```
List<Product> findByOrderByPriceAsc();
Product findTopByOrderByPriceDesc();
List<Product> findTop5ByOrderByPriceDesc();
long countByActiveTrue();
boolean existsByName(String name);
```

SQL ÉQUIVALENT

```
ORDER BY price ASC
ORDER BY price DESC LIMIT 1
ORDER BY price DESC LIMIT 5
SELECT COUNT(*) FROM product WHERE active=true
```

2. Opérations Modificatrices (Suppression)

Les requêtes de type suppression dérivée requièrent d'être enveloppées dans une transaction active.

```
@Transactional
void deleteByName(String name);
```

3. Requêtes Customisées avec @Query (JPQL & Native SQL)

JPQL (Java Persistence Query Language) cible le modèle objet (les entités), tandis que l'argument `nativeQuery = true` permet d'exécuter du SQL brut rattaché à la syntaxe de votre base de données.

Requêtes JPQL (Objet)

```
@Query("SELECT p FROM Product p WHERE p.name = :name")
List<Product> findProductByName(@Param("name") String name);

@Query("SELECT p FROM Product p WHERE p.price > :price")
List<Product> expensiveProducts(@Param("price") BigDecimal price);

@Query("SELECT p FROM Product p WHERE LOWER(p.name) LIKE LOWER(CONCAT('%', :keyword, '%'))")
List<Product> search(@Param("keyword") String keyword);

@Query("SELECT COUNT(p) FROM Product p WHERE p.active = true")
Long countActiveProducts();

@Query("SELECT p.name FROM Product p")
List<String> findAllNames();
```

Requêtes SQL Natives (Brut)

```
// Utile pour les fonctions spécifiques au SGBD
@Query(
    value = "SELECT * FROM product WHERE stock_quantity < :minStock",
    nativeQuery = true
)
List<Product> findLowStockNative(@Param("minStock") Integer minStock);

// Requête de mise à jour / modification personnalisée
@Transactional
@Query("UPDATE Product p SET p.active = false WHERE p.stockQuantity = 0")
void deactivateOutOfStockProducts();
```

4. Pagination et Tri Dynamique

Pour la scalabilité de vos applications, l'utilisation de l'interface Pageable permet d'extraire des sous-ensembles précis de données.

```
// Dans l'interface Repository
Page<Product> findAll(Pageable pageable);

// Dans votre couche Service (Utilisation)
Pageable pageRequest = PageRequest.of(
    0, // Index de la page demandée (commence à 0)
    10, // Nombre d'éléments par page
    Sort.by("price").descending() // Tri dynamique
);

Page<Product> productPage = productRepository.findAll(pageRequest);
List<Product> products = productPage.getContent(); // Éléments de la page actuelle
long totalElements = productPage.getTotalElements(); // Total absolu en BDD
int totalPages = productPage.getTotalPages(); // Nombre total de pages disponibles
```

5. Mots-clés Courants (Guide de Référence Rapide)

Mot-clé JPA	Exemple de Méthode	Prédicat SQL Traduit
And / Or	findByLastnameAndFirstname	where x.lastname = ?1 and x.firstname = ?2
Between	findByPriceBetween	where x.price between ?1 and ?2

Mot-clé JPA	Exemple de Méthode	Prédicat SQL Traduit
LessThan / GreaterThan	<code>findByPriceLessThan</code>	<code>where x.price < ?1</code>
Containing / Like	<code>findByNameContaining</code>	<code>where x.name like %?1%</code>
In	<code>findByBrandIn(Collection<String> b)</code>	<code>where x.brand in (?1)</code>

Ordre d'apprentissage recommandé

- 1. CrudRepository / JpaRepository** : Méthodes natives indispensables (save, findById, delete).
- 2. findByName** : Premiers pas avec les requêtes basées sur le nommage par attribut unique.
- 3. findByNameAndPrice** : Gestion de la logique conditionnelle combinée (AND/OR).
- 4. GreaterThan / LessThan / Between** : Filtres de comparaison indispensables pour les prix et les dates.
- 5. Containing / StartingWith** : Recherche et auto-complétion sur les chaînes de caractères.
- 6. OrderBy** : Tri et structuration des résultats directement par le SGBD.
- 7. Top / First** : Limitation du nombre de lignes ramenées (ex: Top 5).
- 8. Pageable / Page** : Pagination robuste indispensable pour la production.
- 9. @Query JPQL** : Écriture de requêtes complexes sur-mesure orientées objet.
- 10. Native Query** : Optimisation avancée et manipulation de fonctionnalités de bas niveau spécifiques au SGBD.